Python Cookbook

pip, environnement virtuel et import 3
- Installer une librairie 3
- importer une librairie 3
- Sauvegarder les version de bibliothèques 4
- Créer un environnement virtuel et installer une configuration pré installée 4
- Bonnes pratiques 5
- Architecture d'une application python 5
- .gitignore 6

types de données 6
- Types de données 7
- opérations numérique 8
- opérations sur les chaînes de caractères 9

Boucles et conditions 11
- Opération logique 11
- Boucles 12
- Conditions 12
  - Opérateur 13
  - If 13
- Astuces de boucles, de print et d'indexation 14
  - Parcourir deux liste en même temps avec un for 14
  - Parcourir une liste avec enumerate() 14
  - Tout ensemble 15

List, tuples et dictionnaire : Initiation 15
- List 15
  - Création 15
  - Opérations de base sur les listes 16
  - Ajouter des éléments : 16
  - Supprimer des éléments : 17
  - Autre 17
  - Recherche d'un élément : 18

Tuple 19
Création 19
Opérations sur les tuples 19
Utilité 20
Dictionnaire 20
Création 20
Opération de base 21
La compréhension de liste 22
Print : Astuces 23
Fonctions 24
Définition 24
Attention aux types 24
Typage 25
Exception 25
Gérer les erreurs 25
Les exceptions communes 26
documentation docstring 26
Exécution de script 27
Modules et modèles 27
Programmation Orienté Objet (POO) 28
Définition 29
Héritage 29
Encapsulation 30
Les décorateurs 31

pip, environnement virtuel et import

Installer une librairie

pip (Pip Installs Packages) est l'outil standard pour installer des bibliothèques Python. Pour installer une bibliothèque, utilisez la commande pip install nom_de_la_bibliothèque.

```
pip install requests
```

Cette commande télécharge et installe la dernière version de la bibliothèque requests depuis le Python Package Index (PyPI).

importer une librairie

Une fois installée, une bibliothèque peut être utilisée dans un script Python via le mot-clé import.

```
import numpy

nombre_aleatoire = numpy.random.rand()

print(nombre_aleatoire)
```

Il est aussi possible d'importer des parties spécifiques d'une bibliothèque ou de lui attribuer un alias.

```
from datetime import datetime as dt

aujourdhui = dt.now()

print(aujourdhui)
```

Sauvegarder les version de bibliothèques

Pour assurer la reproductibilité d'un projet, il est crucial de sauvegarder les versions des bibliothèques utilisées. La commande
`pip freeze > requirements.txt`
génère un fichier requirements.txt listant toutes les bibliothèques installées et leurs versions.

```
pip freeze > requirements.txt
```

Ce fichier peut ensuite être utilisé pour réinstaller les mêmes versions sur un autre environnement.

```
pip install -r requirements.txt
```

Créer un environnement virtuel et installer une configuration pré installée

Les environnements virtuels permettent d'isoler les dépendances d'un projet. Pour en créer un, utilisez venv (ou virtualenv pour les versions antérieures de Python).

```
python -m venv .venv
```

Activez l'environnement :

- Sous Windows :

```
.venv\Scripts\activate
.venv/Scripts/Activate.ps1 # powershell
```

- Sous macOS et Linux :

```
source .venv/bin/activate
```

Une fois activé, vous pouvez installer les bibliothèques nécessaires, soit manuellement avec pip install, ou comme vu précédemment, depuis un fichier de configuration tel que requirements.txt. Ainsi, les dépendances d'un projet n'interfèrent pas avec celles d'autres projets.

Bonnes pratiques

Architecture d’une application python

Une bonne architecture facilite la maintenance et l'évolutivité. Voici quelques principes clés :

Modularité : Diviser l'application en modules ou packages logiques.

Séparation des préoccupations : Organiser le code en couches (présentation, logique métier, données).

Conventions de nommage : Utiliser des noms clairs et cohérents pour les variables, fonctions et classes (PEP 8).

Gestion des dépendances : Utiliser un environnement virtuel et un fichier requirements.txt.

Tests unitaires et d'intégration : Assurer la qualité du code avec des tests automatisés.

Documentation et procédure d’installation : Utiliser un fichier README.md pour la documentation de l’application.

```
mon_projet/
│
├── mon_module/
│   ├── __init__.py
│   ├── logique_metier.py
│   └── donnees.py
│
├── tests/
│   ├── __init__.py
│   ├── test_logique_metier.py
│   └── test_donnees.py
│
├── main.py
├── requirements.txt
├── .gitignore
└── README.md
```

.gitignore

Le fichier .gitignore permet d'indiquer à Git les fichiers et dossiers à ne pas suivre. C'est crucial pour éviter de versionner des fichiers temporaires, des données sensibles ou des environnements virtuels.

A minima on y met:

```
.venv
.env
```

Qui sont respectivement l'environnement virtuel et les variables d'environnements. Cependant on y ajoute fréquemment:

`__pycache__/, *.pyc, *.pyo, *.pyd` : qui sont les fichiers compilés Python et qui n'ont pas besoin d'être versionnés.

`*.log, *.tmp, *.swp` : qui sont les fichiers temporaires et de journalisation.

Types de données

Python offre une variété de types de données pour représenter différentes sortes d'informations :

Nombres entiers (int) : pour les nombres entiers positifs ou négatifs.

```
age = 30
nombre_negatif = -5
```

Nombres à virgule flottante (float) : pour les nombres réels avec une partie décimale.

```
taille = 1.75
pi = 3.14159
```

Chaînes de caractères (str) : pour le texte.

```
nom = "Jean"
message = 'Bonjour, le monde!'
```

Booléens (bool) : pour les valeurs vrai ou faux.

```
est_majeur = True
est_pret = False
```

Listes (list) : pour les séquences ordonnées d'éléments.

```
nombres = [1, 2, 3, 4, 5]
fruits = ["pomme", "banane", "orange"]
```

Tuples (tuple) : comme les listes, mais immuables.

```
coordonnees = (10, 20)
```

Dictionnaires (dict) : pour les collections de paires clé-valeur.

```
personne = {"nom": "Jean", "age": 30}
```

opérations numérique

Python supporte les opérations arithmétiques classiques :

Addition (+), Soustraction (-), Multiplication (*), Division (/)

`x = 10 + 5  # 15`	`z = 10 * 5  # 50`
`y = 10 - 5  # 5`	`w = 10 / 5  # 2.0`

Division entière (//), Modulo (%), Puissance (**)

```
a = 10 // 3  # 3
b = 10 % 3   # 1
c = 2 ** 3   # 8
```

opérations sur les chaînes de caractères

Concaténation (+)

```
nom = "Jean" + " " + "Dupont"  # "Jean Dupont"
```

Répétition (*)

```
tirets = "-" * 10  # "----------"
```

Accès aux caractères (indexation)

```
mot = "Python"
premiere_lettre = mot[0]  # "P"
derniere_lettre = mot[-1] # "n"
```

Découpage (slicing)

```
sous_mot = mot[1:4]  # "yth"
```

Méthodes de chaînes : `upper()` `lower()` `strip()` `split()`

```
mot_en_majuscules = mot.upper()
# "PYTHON"
mots = "Bonjour le monde".split()
# ["Bonjour", "le", "monde"]
```

Inversion d'une séquence

```
texte = "Python"
texte_inverse = texte[::-1]
print(texte_inverse)  # Affiche "nohtyP"
nombres = [1, 2, 3, 4, 5]
```

```
nombres_inverses = nombres[::-1]
print(nombres_inverses)  # Affiche [5, 4, 3, 2, 1]
```

Sélection d'éléments avec un pas de 2

```
texte = "Python"
texte_pas_de_deux = texte[::2]
print(texte_pas_de_deux)  # Affiche "Pto"
nombres = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
nombres_pas_de_deux = nombres[::2]
print(nombres_pas_de_deux)  # Affiche [0, 2, 4, 6, 8]
```

Combinaisons et variations

Le slicing offre une grande flexibilité. Vous pouvez combiner les valeurs de début, fin et pas pour obtenir des résultats plus complexes.

`[1:5:2]` : Commence à l'indice 1, se termine à l'indice 5 (non inclus), et prend chaque deuxième élément.

`[::  -2]` : Parcours la séquence à l'envers, en prenant chaque deuxième élément.

Boucles et conditions

Opération logique

Les opérations logique en python se parle, elles n'utilisent pas les traditionnelles & ou | comme en C mais utilisent des mots (and, or).

and (et logique) :

Renvoie True si toutes les conditions sont vraies.
Exemple : if a > 0 and b < 10:

or (ou logique) :

Renvoie True si au moins une des conditions est vraie.
Exemple : if a == 0 or b == 0:

not (négation logique) :

Inverse la valeur d'une condition (transforme True en False et vice-versa).
Exemple : if not est_vide:

is (identité) :

Vérifie si deux variables font référence au même objet en mémoire.
À ne pas confondre avec ==, qui vérifie l'égalité des valeurs.

is not (négation de l'identité) :

Vérifie si deux variables ne font pas référence au même objet.
Exemple : if a is not b:

in (appartenance) :

Vérifie si une valeur est présente dans une séquence (liste, tuple, chaîne, etc.).
Exemple : if "a" in "banana":

not in (négation de l'appartenance) :

Vérifie si une valeur n'est pas présente dans une séquence.
Exemple : if "z" not in "banana":

Boucles

Python propose deux types de boucles : for et while.

Boucle for :

- Utilisée pour itérer sur une séquence (liste, tuple, chaîne, etc.) ou un itérable.
- Exécute le bloc de code pour chaque élément de la séquence.

```
fruits = ["pomme", "banane", "orange"]
for fruit in fruits:
    print(fruit)

for i in range(5):  # boucle de 0 à 4
    print(i)
```

Boucle while :

- Exécute le bloc de code tant qu'une condition est vraie.
- Attention à éviter les boucles infinies !

```
compteur = 0
while compteur < 5:
    print(compteur)
    compteur += 1
```

Conditions

Opérateur

==	égal à	!=	différent de
>	strictement supérieur à	>=	supérieur ou égal à
<	strictement inférieur à	<=	inférieur ou égal à

If

Les conditions permettent d'exécuter un bloc de code uniquement si une condition est remplie. Python utilise if, elif (sinon si) et else (sinon) pour les conditions.

```
age = 20
if age >= 18:
    print("Vous êtes majeur.")
elif age >= 16:
    print("Vous êtes presque majeur.")
else:
    print("Vous êtes mineur.")
```

Astuces de boucles, de print et d'indexation

Parcourir deux listes en même temps avec un for

La fonction intégrée zip() est la méthode la plus courante et la plus élégante pour parcourir deux ou plusieurs listes en parallèle. Elle crée un itérateur qui agrège les éléments de chaque liste en tuples.

```
    noms = ["Alice", "Bob", "Charlie", "David"]
    ages = [25, 30, 22]

    for nom, age in zip(noms, ages):
        print(f"{nom} a {age} ans.")
```

Dans cet exemple, seuls les 3 premiers couples seront pris en compte.

Parcourir une liste avec enumerate()

```
fruits = ["pomme", "banane", "orange"]

for indice, fruit in enumerate(fruits):
    print(f"L'indice {indice} contient : {fruit}")

# Résultat :
# L'indice 0 contient : pomme
# L'indice 1 contient : banane
# L'indice 2 contient : orange
```

Tout ensemble

```
noms = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 22]
villes = ["Paris", "Lyon", "Marseille"]

for indice, (nom, age, ville) in enumerate(zip(noms, ages, villes)):
    print(f"L'indice {indice}: {nom} a {age} ans et vit à {ville}.")

# Résultat :
# L'indice 0: Alice a 25 ans et vit à Paris.
# L'indice 1: Bob a 30 ans et vit à Lyon.
# L'indice 2: Charlie a 22 ans et vit à Marseille.
```

List, tuples et dictionnaire : Initiation

List

Les listes sont des séquences ordonnées et modifiables d'éléments. Elles sont créées en utilisant des crochets [] et les éléments sont séparés par des virgules.

Création

Liste vide :

```
    ma_liste = []
```

Liste avec des éléments de différents types :

```
    ma_liste = [1, "Python", 3.14, True]
```

Liste de listes (liste imbriquée) :

```
ma_liste = [[1, 2, 3], ["a", "b", "c"]]
```

Création d'une liste à partir d'un itérable :

```
ma_liste = list(range(5))  # [0, 1, 2, 3, 4]
```

Opérations de base sur les listes

Accéder aux éléments (indexation) :

```
ma_liste = ["pomme", "banane", "orange"]
print(ma_liste[0])  # Affiche "pomme"
print(ma_liste[-1]) # Affiche "orange"
```

Modifier des éléments :

```
ma_liste[1] = "kiwi"
print(ma_liste)
# Affiche ["pomme", "kiwi", "orange"]
```

Ajouter des éléments :

append() : Ajoute un élément à la fin de la liste.

```
ma_liste.append("mangue")
```

extend() : Ajoute tous les éléments d'une autre liste.

```
ma_liste.extend(["fraise", "ananas"])
```

insert() : Insère un élément à une position donnée.

```
ma_liste.insert(1, "poire")
```

Supprimer des éléments :

remove() : Supprime la première occurrence d'un élément.

```
ma_liste.remove("banane")
```

pop() : Supprime et renvoie l'élément à une position donnée (ou le dernier élément par défaut).

```
element_supprime = ma_liste.pop(0)
```

del : Supprime un élément ou une tranche de la liste.

```
del ma_liste[0] # supprime l'élément à l'index 0
del ma_liste[0:2] # supprime de l'index 0 à 1
```

Autre

Longueur d'une liste :

```
print(len(ma_liste))
```

Découpage (slicing) :

```
sous_liste = ma_liste[1:3]
```

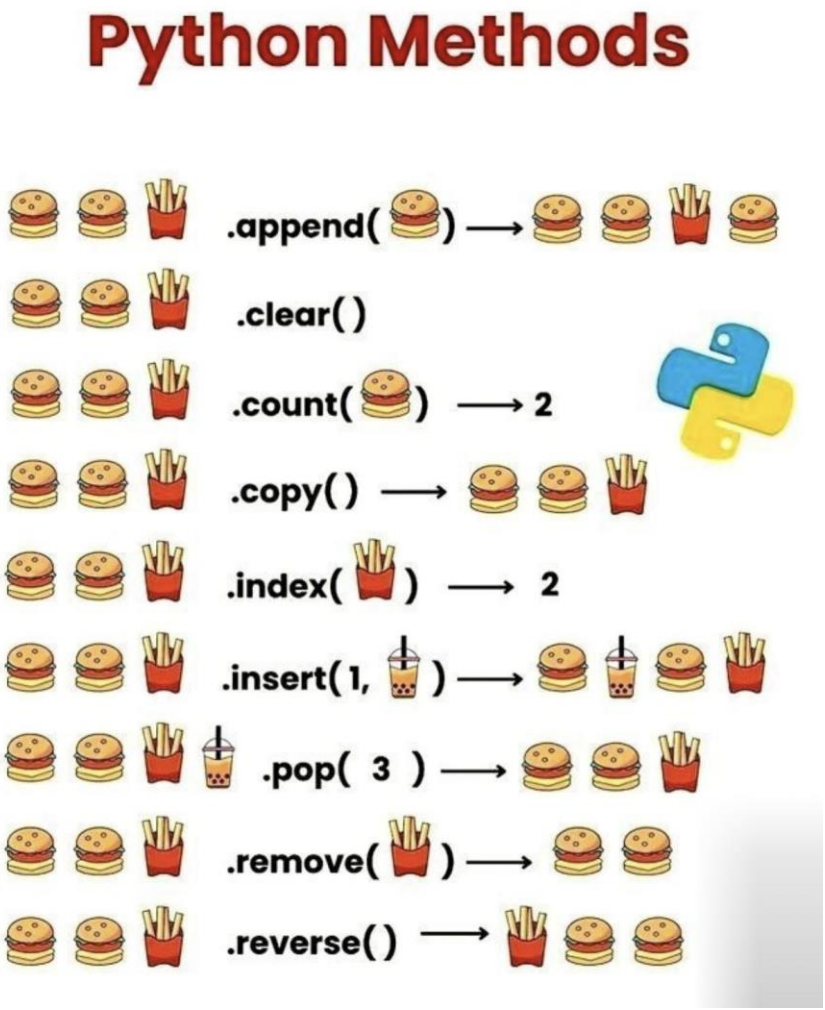
Recherche d'un élément :

index() : Renvoie l'indice de la première occurrence d'un élément.

```
indice_banane = ma_liste.index("banane")
```

count() : Renvoie le nombre d'occurrences d'un élément.

```
nombre_pommes = ma_liste.count("pomme")
```



Tuple

Les tuples sont des séquences ordonnées et immuables d'éléments. Ils sont créés en utilisant des parenthèses () et les éléments sont séparés par des virgules.

Création

Tuple vide

```
mon_tuple = ()
```

Tuple avec des éléments de différents types

```
mon_tuple = (1, "Python", 3.14, True)
```

Tuple à un seul élément

```
mon_tuple = (42,)
```

Opérations sur les tuples

Accès aux éléments et slicing

```
mon_tuple = (1, 2, 3)
print(mon_tuple[0])  # Affiche 1
sous_tuple = mon_tuple[1:3]  # (2, 3)
```

Longueur d'un tuple

```
print(len(mon_tuple))
```

Concaténation de tuples

```
tuple1 = (1, 2)
tuple2 = (3, 4)
tuple_concatene = tuple1 + tuple2  # (1, 2, 3, 4)
```

Utilité

Immuabilité : Les tuples ne peuvent pas être modifiés après leur création. Cela garantit que les données restent constantes tout au long de l'exécution du programme.

Performance : Les tuples sont généralement plus rapides que les listes en termes de performance.

Sécurité : Les tuples offrent une certaine sécurité contre les modifications accidentelles de données.

Dictionnaire

Les dictionnaires sont des collections non ordonnées de paires clé-valeur. Ils sont créés en utilisant des accolades {} et des paires clé-valeur qui sont séparées par des virgules.

Création

On peut créer un dictionnaire de différentes manières

```
    mon_dictionnaire = {}


    mon_dictionnaire = {"nom": "Jean", "age": 30,
"ville": "Paris"}


paires = [("nom", "Jean"), ("age", 30), ("ville",
"Paris")]
mon_dictionnaire = dict(paires)


cles = ["nom", "age", "ville"]
valeurs = ["Jean", 30, "Paris"]
mon_dictionnaire = dict(zip(cles, valeurs))
```

De la même manière on peut créer un dictionnaire dans un dictionnaire

Opération de base

Les opérations sont similaires à celles des listes.

Accéder, modifier une valeur

```
print(mon_dictionnaire["nom"])  # Affiche "Jean"

mon_dictionnaire["age"] = 31

mon_dictionnaire["profession"] = "développeur"
```

Supprimer une paire clé-valeur

```
del mon_dictionnaire["ville"]

# Supprime et renvoie la valeur associée à une clé.
age = mon_dictionnaire.pop("age")
```

Vérifier si une clé existe et obtenir toutes les clés

```
"nom" in mon_dictionnaire  # Renvoie True

mon_dictionnaire.keys()  # Renvoie une vue des clés
```

Obtenir les valeurs

```
mon_dictionnaire.values() # Renvoie une vue des
valeurs

mon_dictionnaire.items() # Renvoie une vue des paires
clé-valeur sous forme de tuples

len(mon_dictionnaire)
```

Parcourir un dictionnaire

```
for cle, valeur in mon_dictionnaire.items():
    print(cle, valeur)
```

La compréhension de liste

Une compréhension de liste est une manière concise de créer une nouvelle liste à partir d'une séquence existante (liste, tuple, range, etc.) en appliquant une expression à chaque élément et en filtrant éventuellement les éléments qui ne satisfont pas une condition.

```
carres_pairs= [x**2 for x in range(10) if x % 2 == 0]

print(carres_pairs) # affiche [0, 4, 16, 36, 64]
```

Print : Astuces

Les f-strings permettent d'insérer des expressions Python directement dans des chaînes de caractères, offrant un moyen concis et lisible de formater les sorties:

- **Centrage** (:=^) : Centre la valeur dans un champ de largeur spécifiée, en remplissant les espaces avec le caractère donné.
- **Alignement à droite** (:=>) : Aligne la valeur à droite dans un champ de largeur spécifiée.
- **Alignement à gauche** (:=<) : Aligne la valeur à gauche dans un champ de largeur spécifiée.

```
var1 = "Python"
```

```
var2 = 123

print(f'{var1:=^20} {var2:=^20}')
# Affiche : ======Python====== ========123========

print(f'{var1:=>20} {var2:=<20}')
# Affiche : =============Python 123=================

print(f'{var1:=<20} {var2:=>20}')
# Affiche : Python============= =================123
```

On peut aussi utiliser les opérateurs comme en C ou des formats spécifique au date

```
print(f'{pi:.2f}')  # Affiche 3.14 (2 décimales)
print(f'{now:%Y-%m-%d %H:%M:%S}')
# Affiche actuelle au format YYYY-MM-DD HH:MM:SS
```

Fonctions

Les fonctions permettent de regrouper un bloc de code réutilisable. Elles sont définies à l'aide du mot-clé def.

Définition

```
def nom_fonction(parametre1, parametre2, ...):
    """Documentation de la fonction (docstring)"""
    # Corps de la fonction
    return valeur_de_retour
```

Paramètres par défaut (à placer en dernier argument)

```
def saluer(nom="invité"):
    """Salue une personne."""
    print(f"Bonjour, {nom} !")

saluer()          # Bonjour, invité !
saluer("Alice")   # Bonjour, Alice !
```

Attention aux types

Python est un langage à typage dynamique, ce qui signifie que les types des variables sont vérifiés lors de l'exécution. Cela offre une grande flexibilité, mais peut aussi conduire à des erreurs si les types ne sont pas gérés correctement.

```
def additionner(a, b):
    return a + b
resultat = additionner("1", 2)  # Erreur : TypeError
```

Typage

On peut typer les fonctions bien que cela ne change rien. Ça peut aider les créations de tests automatiques.

```
def additionner(a: int, b: int) -> int:
    return a + b
additionner("to", "ta") # tota
```

Exception

Les exceptions sont des erreurs qui se produisent pendant l'exécution d'un programme. Python offre un mécanisme pour gérer ces exceptions à l'aide des blocs try-except.

Gérer les erreurs

```
try:
    # Code susceptible de lever une exception
except TypeException as variable:
    # Code à exécuter en cas d'exception
finally:
    # Code exécuté toujours (optionnel)
```

Si il n'y a aucune exception de levée

```
try:
    resultat = 10 / 2
except ZeroDivisionError:
    print("Division par zéro !")
else:
    print(f"Le résultat est : {resultat}")
```

Les exceptions communes

Voici quelques erreurs communes en Python :

- **TypeError** : Opération ou fonction appliquée à un type incorrect.
- **ValueError** : Opération ou fonction appliquée à un type correct, mais avec une valeur incorrecte.
- **IndexError** : Index de liste hors limites.
- **KeyError** : Clé de dictionnaire non trouvée.

- **FileNotFoundError** : Fichier non trouvé.
- **ZeroDivisionError** : Division par zéro.
- **ImportError** : Impossible d'importer un module.
- **AttributeError** : Tentative d'accès à un attribut inexistant.
- **NameError** : Variable non définie.

documentation docstring

Les docstrings sont des chaînes de caractères littérales qui apparaissent comme la première déclaration dans la définition d'une fonction, d'une classe, d'une méthode ou d'un module. Elles servent à documenter le code.

```
def calculer_somme(a, b):
    """
    Calcule la somme de deux nombres.

    Args:
        a (int): Le premier nombre.
        b (int): Le deuxième nombre.

    Returns:
        int: La somme de a et b.
    """
```

Exécution de script

`if __name__ == '__main__':`

Cette clause est utilisée pour déterminer si un script Python est exécuté directement ou s'il est importé en tant que module.

```
# mon_script.py
def fonction_utile():
    print("Fonction utile.")


if __name__ == '__main__':
    print("Exécution du script principal.")
    fonction_utile()
```

Cela permet d'éviter que ce code ne soit exécuté lorsque le script est importé en tant que module.

Modules et modèles

Modules : Les modules sont des fichiers Python contenant des définitions de fonctions, de classes et de variables. Ils permettent d'organiser et de réutiliser le code.

Packages : Les packages sont des répertoires contenant des modules Python. Ils permettent de structurer les modules en hiérarchies.

```
mon_package/
├── __init__.py
├── module_a.py
└── module_b.py
```

Les fichiers `__init__.py` sont utilisés pour indiquer à Python qu'un répertoire doit être considéré comme un package.

Import d'un module du package `mon_package`

```
# mon_package/__init__.py
from .module_a import fonction_a
from .module_b import fonction_b
__all__ = ['fonction_a']
```

Dans le fichier `__init__.py`, la variable `__all__` peut être utilisée pour définir explicitement quelles fonctions ou classes doivent être importées lorsque l'instruction `from mon_package import *` est utilisée. Ici la fonction `fonction_b` ne peut pas être importée.

Programmation Orienté Objet (POO)

La POO est un paradigme de programmation qui utilise des objets, qui sont des instances de classes, pour structurer et organiser le code.

Classe : Un modèle ou un plan pour créer des objets. Elle définit les attributs (données) et les méthodes (fonctions) des objets.

Objet (instance) : Une instance spécifique d'une classe. Il contient les données réelles et peut appeler les méthodes définies dans la classe.

Définition

```
class Voiture:
    def __init__(self, marque, modele):
        self.marque = marque
        self.modele = modele

    def demarrer(self):
        print(f"La {self.marque} {self.modele}
démarre.")

ma_voiture = Voiture("Toyota", "supra 4")
ma_voiture.demarrer()
# Affiche "La Toyota supra 4 démarre."
```

Héritage

L'héritage permet de créer de nouvelles classes (sous-classes) à partir de classes existantes (classes parentes ou superclasses). Les sous-classes héritent des attributs et des méthodes des superclasses, et peuvent les modifier ou en ajouter de nouveaux.

```
class Vehicule:
    def __init__(self, marque):
        self.marque = marque

class Voiture(Vehicule):
    def __init__(self, marque, modele):
        super().__init__(marque)
        self.modele = modele
```

Encapsulation

L'encapsulation est la manière d'accéder ou de changer la valeur d'une variable. On crée des méthodes pour le faire à notre place.

```
class Personne:
    def __init__(self, nom, age):
        self._nom = nom  # attribut 'protégé'
        self._age = age  # attribut 'protégé'

    @property
    def nom(self):
        """ Getter pour le nom. """
        return self._nom

    @property
    def age(self):
        """ Getter pour l'âge. """
        return self._age

    @age.setter
    def age(self, valeur):
        """ Setter pour l'âge, avec validation. """
        if valeur < 0:
            raise ValueError("L'âge doit être
positif.")
        self._age = valeur

p = Personne("Alice", 30)
```

```
# Utilisation des getters (comme des attributs)
print(p.nom)
print(p.age)
# Utilisation du setter
p.age = 35
print(p.age)
# Tentative de définir un âge négatif (lève
ValueError)
# p.age = -5
```

Les décorateurs

Un décorateur est une fonction qui prend une autre fonction en argument, ajoute des fonctionnalités à cette fonction, et la renvoie.

```
def mon_decorateur(fonction):
    def fonction_enveloppe():
        # Code avant l'appel à la fonction
        print("Quelque chose se passe avant la
fonction.")
        fonction()
        # Code après l'appel à la fonction
        print("Quelque chose se passe après la
fonction.")
    return fonction_enveloppe

@mon_decorateur
def dis_bonjour():
    print("Bonjour !")
```

```
dis_bonjour()
```

Exemple de décorateur qui mesure le temps d'exécution d'une fonction.

```
import time

def mesurer_temps(fonction):
    """Décorateur pour mesurer le temps d'exécution
d'une fonction."""
    def fonction_enveloppe(*args, **kwargs):
        debut = time.time()
        resultat = fonction(*args, **kwargs)
        fin = time.time()
        print(f"La fonction '{fonction.__name__}' a
pris {fin - debut:.4f} secondes.")
        return resultat
    return fonction_enveloppe

@mesurer_temps
def fonction_longue(n):
    """Fonction qui prend du temps à s'exécuter."""
    somme = 0
    for i in range(n):
        somme += i
    return somme

resultat = fonction_longue(1000000)
print(f"Résultat : {resultat}")
```